

Agentes Inteligentes

Prof. Agustín Gravano

Universidad ORT Uruguay - 1^{er} Semestre 2020

7. Redes Neuronales Artificiales

Bibliografía:

- Sutton & Barto, “Reinforcement Learning”, MIT Press, 2018. Sección 9.7.
- Goodfellow, Bengio & Courville, “Deep Learning”, MIT Press, 2016. Capítulos 6 y 9.
(Disponible online: <http://www.deeplearningbook.org>)

Enlaces:

- Videos: [Single-layer ANN](#), [Multilayer ANN](#)
- Demo: [TensorFlow Playground](#).

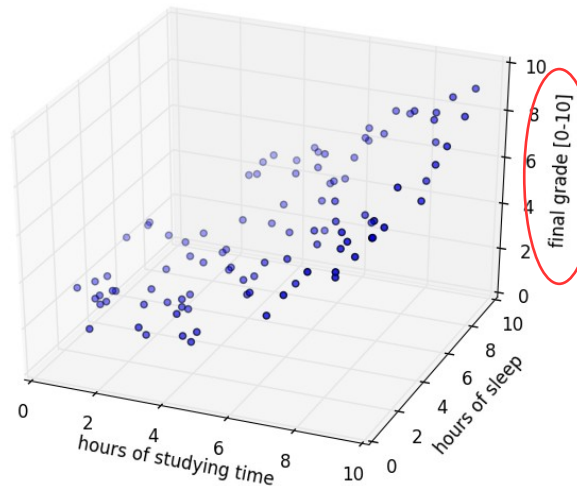
Problema de Regresión

- La variable dependiente Y es **numérica**:

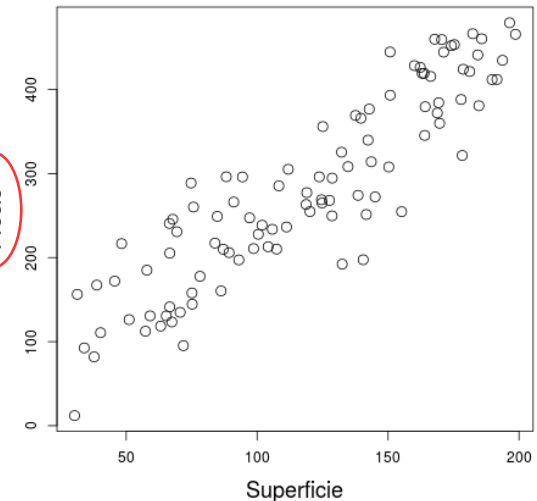
$$Y \approx f(X)$$

- Ejemplos:

- Ventas en función de la inversión en publicidad.
- Edad de un usuario a partir de sus tweets.
- Tiempo de demora de un vuelo.
- Nota en un examen:
- Precio de un departamento:



(2 atributos)



(1 atributo)

Problema de Regresión

- Consiste en predecir una respuesta **numérica** Y en base a p atributos $X_1, X_2, \dots, X_p$.

$$Y \approx f(X_1, X_2, \dots, X_p)$$

f es **desconocida**; el **error** puede deberse a errores de medición o a la ausencia de otros atributos que expliquen la variación de Y .

- Un algoritmo de regresión ajusta un modelo $\hat{f}$ a los datos.
- Después sirve para predecir el valor de una nueva instancia:

$$\hat{y} = \hat{f}(x_1, x_2, \dots, x_p)$$

- Evaluación: **error cuadrático medio** (MSE) y otras métricas.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \underbrace{(y^{(i)} - \hat{y}^{(i)})}_{\text{error}}^2$$

Regresión Lineal Simple

- Consiste en predecir una respuesta numérica Y en base a una única variable predictora X_1 , **suponiendo una relación lineal**.

$$Y \approx \beta_0 + \beta_1 \cdot X_1$$

β_0 y β_1 son **desconocidos**; el **error** ahora también puede deberse a que la relación no sea realmente lineal.

- Ejemplo:

$$\textit{Precio} \approx \beta_0 + \beta_1 \cdot \textit{Superficie}$$

Ordenada al origen (*intercept*)

Pendiente (*slope*)

- Ajustamos los parámetros del modelo lineal (β_0 y β_1) a los datos de entrenamiento. Después puede usarse para predecir:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 \cdot x_1$$

Valor predicho para Y
cuando $X_1 = x_1$

Estimación de β_0

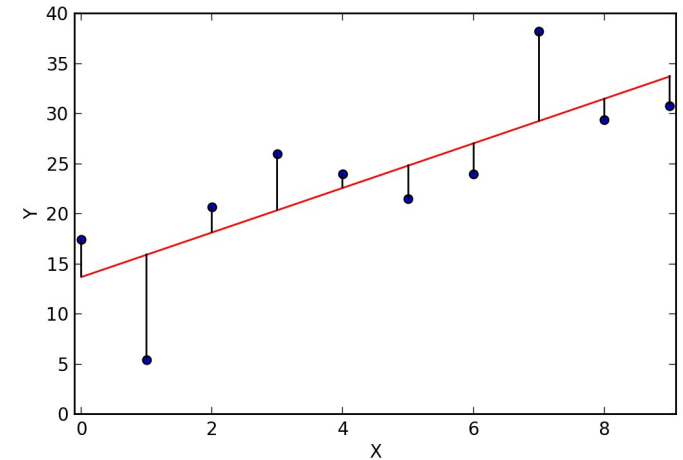
Estimación de β_1

Nueva instancia

Estimación de coeficientes

- Residuo o error de predicción:

$$e^{(i)} = y^{(i)} - \hat{y}^{(i)}$$



- Residual sum of squares:

$$\text{RSS} = \sum_{i=1}^n e^{(i)2} = \sum_{i=1}^n \left(y^{(i)} - \hat{\beta}_0 - \hat{\beta}_1 \cdot x_1^{(i)} \right)^2$$

Estimación de coeficientes

$$\text{RSS} = \sum_{i=1}^n \left(y^{(i)} - \hat{\beta}_0 - \hat{\beta}_1 \cdot x_1^{(i)} \right)^2$$

- Para estimar los coeficientes, buscamos minimizar RSS (lo cual es equivalente a minimizar MSE):

$$\frac{\delta \text{RSS}}{\delta \hat{\beta}_0} = 0$$

$$\frac{\delta \text{RSS}}{\delta \hat{\beta}_1} = 0$$

- Con pocas cuentas, llegamos a estas expresiones:

$$\hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \cdot \bar{X}_1$$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_1^{(i)} - \bar{X}_1)(y^{(i)} - \bar{Y})}{\sum_{i=1}^n (x_1^{(i)} - \bar{X}_1)^2} = \frac{\text{Cov}[X_1, Y]}{\text{Var}[X_1]}$$

Predicción de nuevos valores

- Estimados los coeficientes, los podemos usar para predecir:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 \cdot x_1$$

- En nuestro ejemplo, podríamos predecir el precio de un departamento de 75m²:

$$\widehat{Precio} = \hat{\beta}_0 + \hat{\beta}_1 \cdot 75$$

Regresión Lineal Múltiple

- Consiste en predecir una respuesta numérica Y en base a múltiples variables predictoras $X_1, X_2, \dots, X_p$, suponiendo una relación lineal.

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

- MSE, RSS, TSS, R^2 se definen en forma análoga que para la regresión lineal simple.
- Los coeficientes también se estiman en forma análoga.

Métricas para evaluar modelos

- **MSE**: Error cuadrático medio

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- **R^2** : Proporción de la variabilidad explicada por el modelo.

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}}$$

donde RSS: Variabilidad no explicada por el modelo

TSS: Variabilidad total de los datos

$$\text{RSS} = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

$$\text{TSS} = \sum_{i=1}^n (y^{(i)} - \bar{Y})^2$$

$R^2 \rightarrow 0$ cuando el modelo explica poco de la variabilidad de los datos.

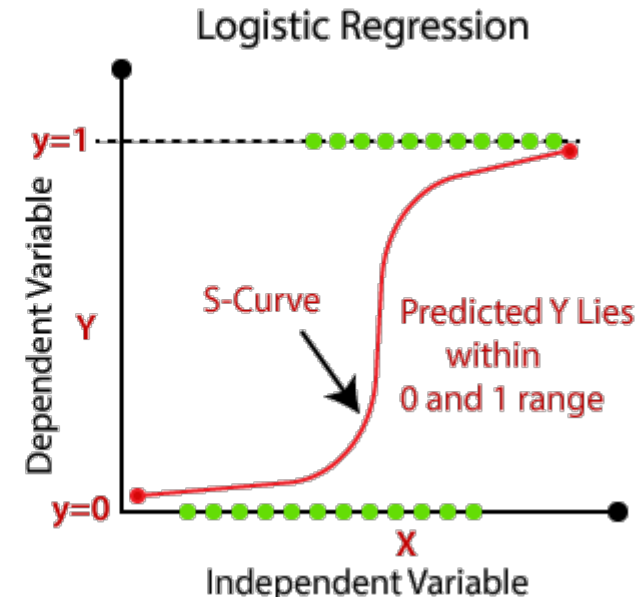
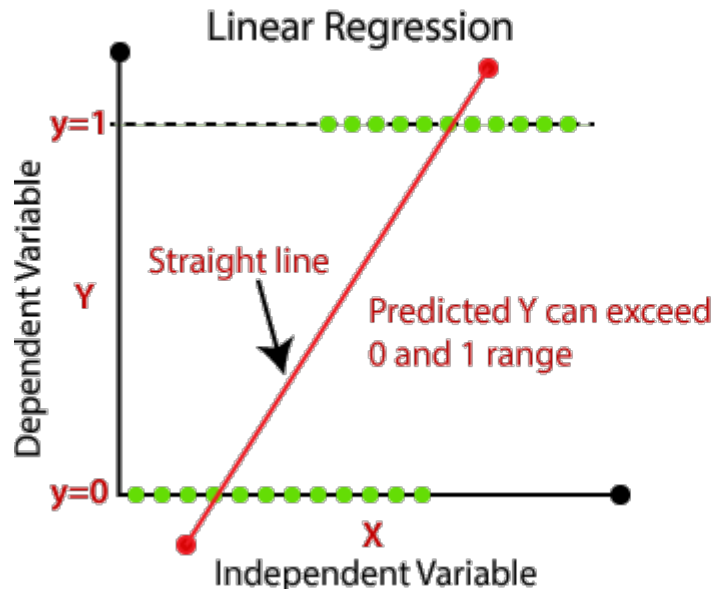
$R^2 \rightarrow 1$ cuando el modelo explica mucho de la variabilidad de los datos.

Regresión Lineal vs. Logística

Supongamos que queremos predecir una probabilidad.

Ejemplo: Predecir la probabilidad de que un estudiante juegue al basket según su altura.

X: altura, Y: juega (sí/no)



La regresión lineal tiene problemas para modelar probabilidades.

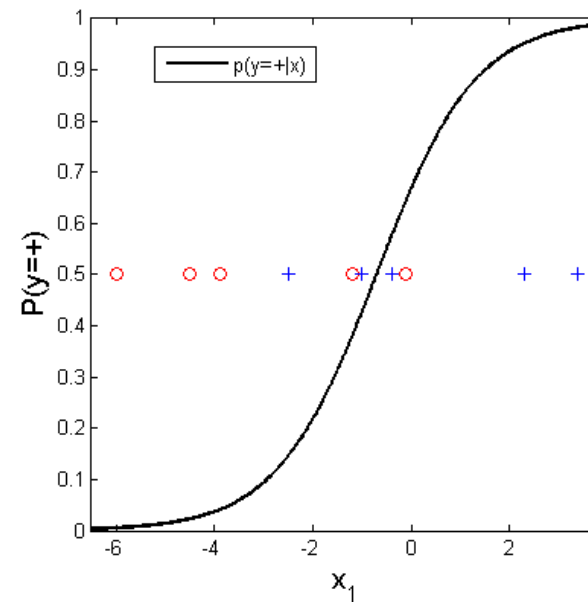
Mucho más adecuada es la **regresión logística**.

Regresión Logística

$$Y \approx f(\beta_0 + \beta_1 \cdot X_1) \quad \text{donde } f(t) = \frac{e^t}{1 + e^t} \quad \text{función logística}$$

La función logística es equivalente a la **función sigmoidea**: $\sigma(t) = \frac{1}{1 + e^{-t}}$

$$Y \approx \sigma(\beta_0 + \beta_1 \cdot X_1)$$



¿Cómo ajustamos los coeficientes β_0 y β_1 a un dataset?

Regresión Logística: Ajuste

Estimamos el error de predicción con la **función de pérdida de entropía cruzada** (*cross-entropy loss*):

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -y^{(i)} \cdot \log \hat{y}^{(i)} - (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)})$$

Definimos el costo J de un modelo θ :

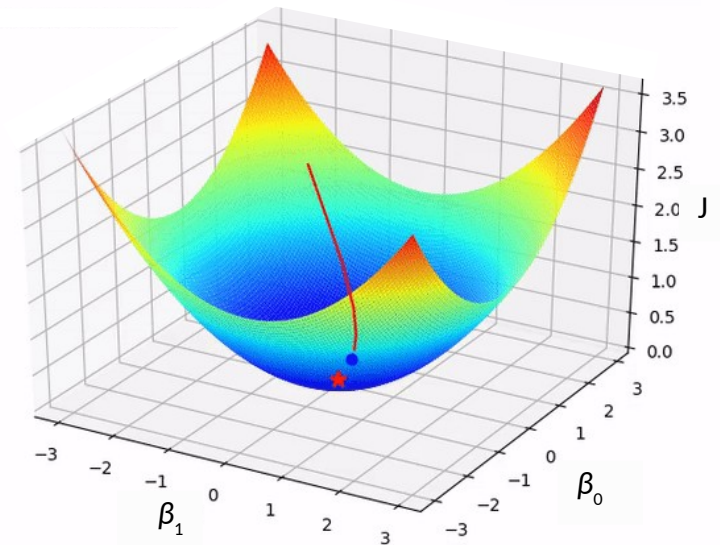
$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Actualizamos β_0 y β_1 iterativamente, con descenso por el gradiente:

$$\nabla J \equiv \left[\frac{\partial J}{\partial \beta_0}, \frac{\partial J}{\partial \beta_1} \right]$$

tasa de aprendizaje

$$\begin{cases} \beta_0^{[t+1]} \leftarrow \beta_0^{[t]} - \eta \frac{\partial J}{\partial \beta_0} \\ \beta_1^{[t+1]} \leftarrow \beta_1^{[t]} - \eta \frac{\partial J}{\partial \beta_1} \end{cases}$$

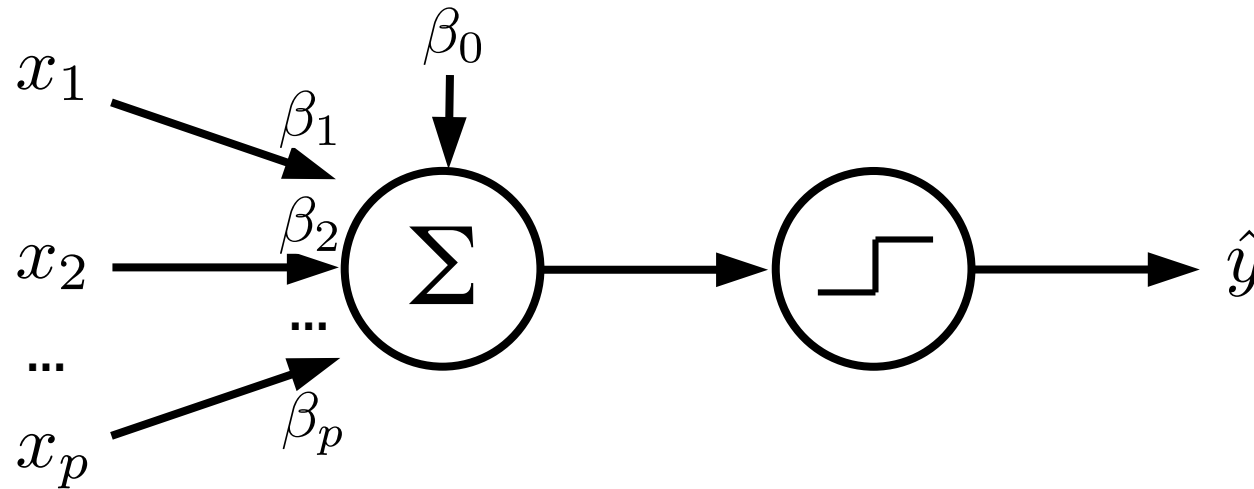


Todo esto generaliza a p variables predictoras:

$$Y \approx \sigma(\beta_0 + \beta_1 \cdot X_1 + \dots + \beta_p \cdot X_p)$$

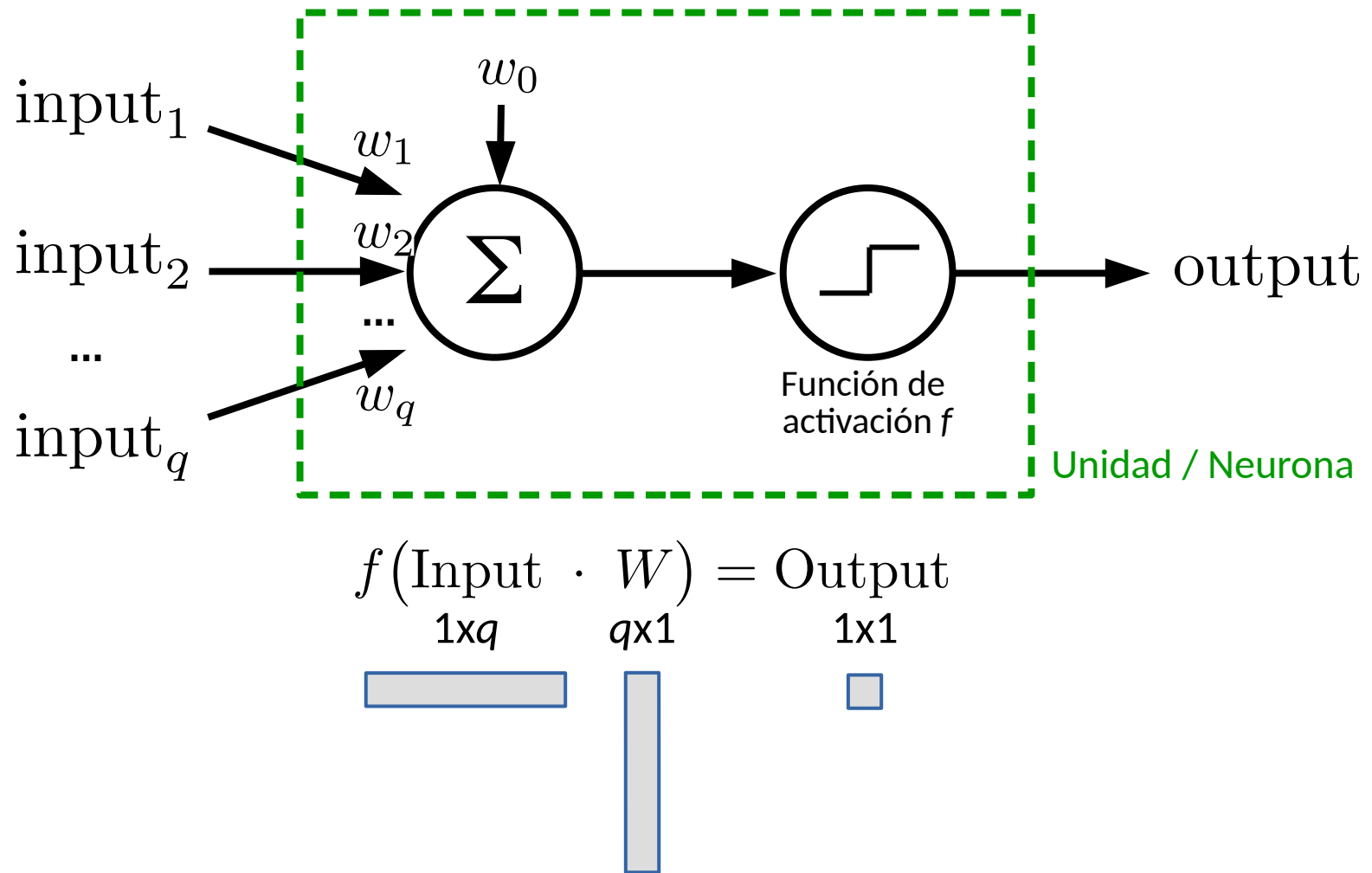
$$\beta_j^{[t+1]} \leftarrow \beta_j^{[t]} - \eta \frac{\partial J}{\partial \beta_j}$$

Perceptrón Simple = Regresión Logística



$$\hat{y} = \sigma \left(\sum_{j=0}^p \beta_j \cdot x_j \right) \quad (\text{Def: } x_0 \equiv 1)$$

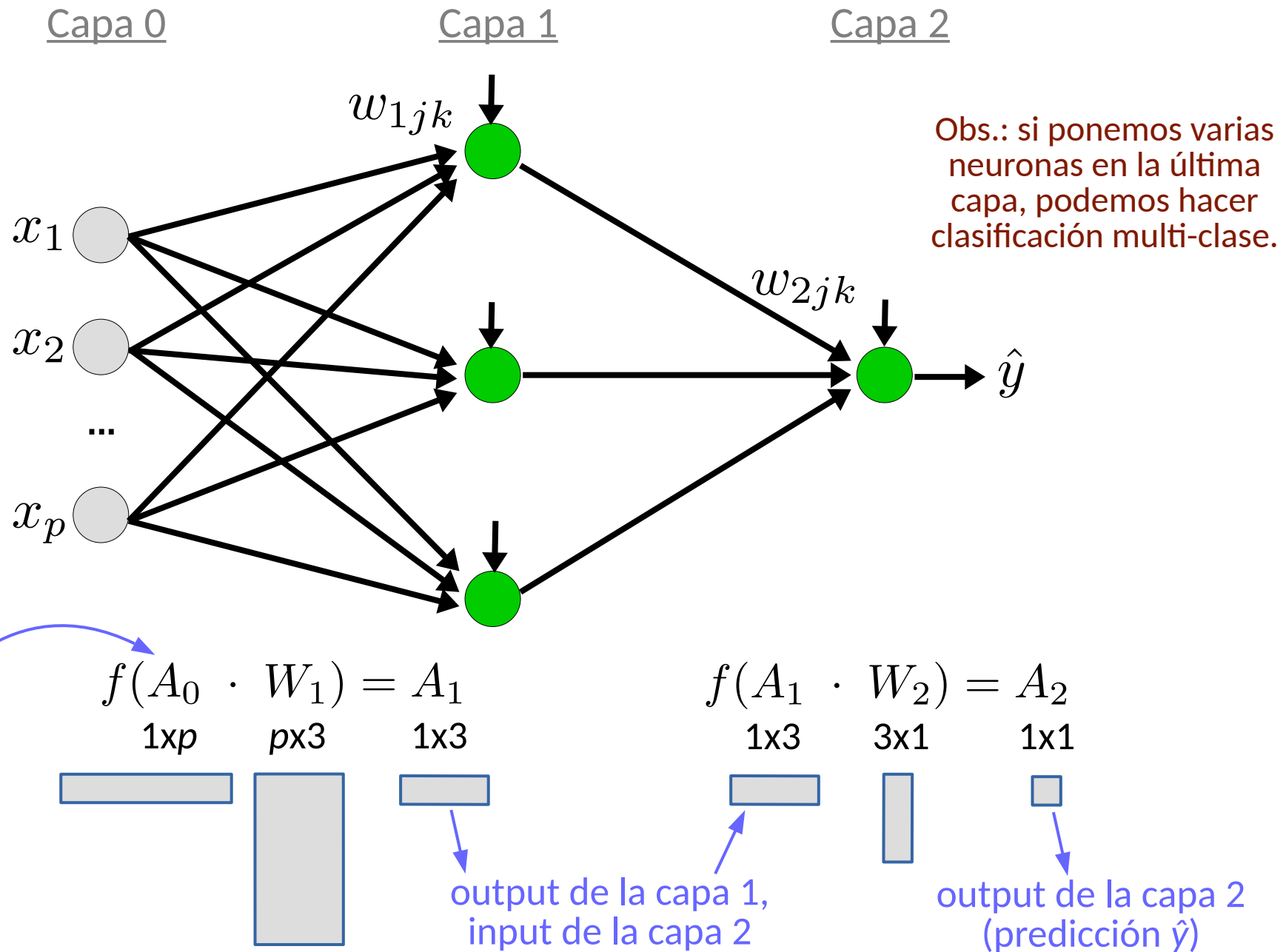
Perceptrón Simple



Funciones de activación: sigmoidea; identidad ($f(x)=x$); escalón ($f(x)=0$ si $x<0$, 1 si $x\geq 0$); tangente hiperbólica ($f(x)=\tanh(x)$); ReLU ($f(x)=0$ si $x<0$, x si $x\geq 0$); etc.

Obs: Por simplicidad, en las matrices se omiten las ordenadas al origen (w_0).

Perceptrón Multicapa



Perceptrón Multicapa: Ajuste

Costo del modelo:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

función de pérdida

$\left\{ \begin{array}{l} \text{cross-entropy loss, para clasificación} \\ \text{error cuadrático, para regresión} \end{array} \right.$

Igual que en regresión logística, actualizamos los parámetros de θ iterativamente, con **descenso por el gradiente**:

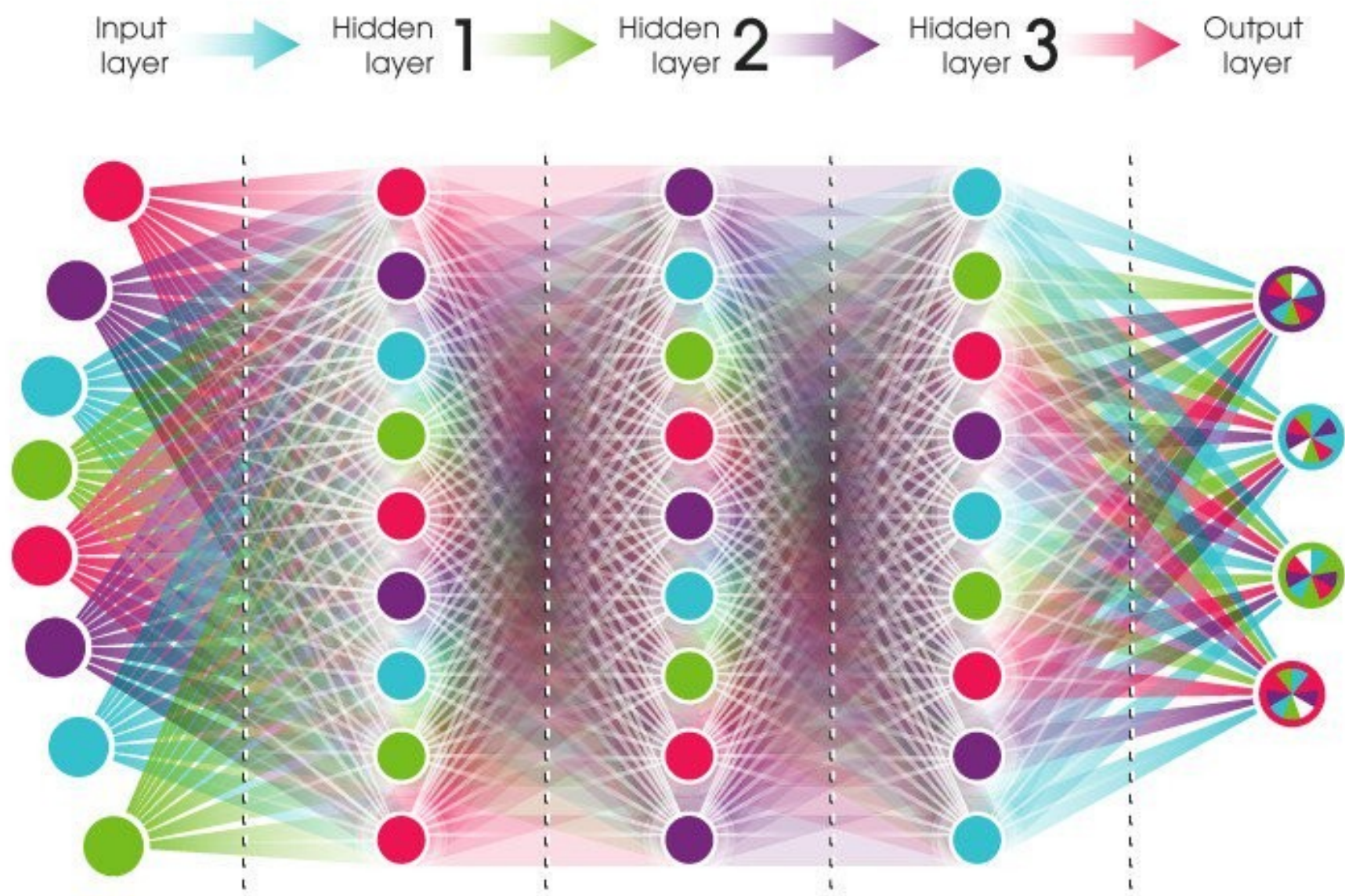
$$w_{ijk}^{[t+1]} \leftarrow w_{ijk}^{[t]} - \eta \frac{\partial J(\theta)}{\partial w_{ijk}}$$

Error back-propagation

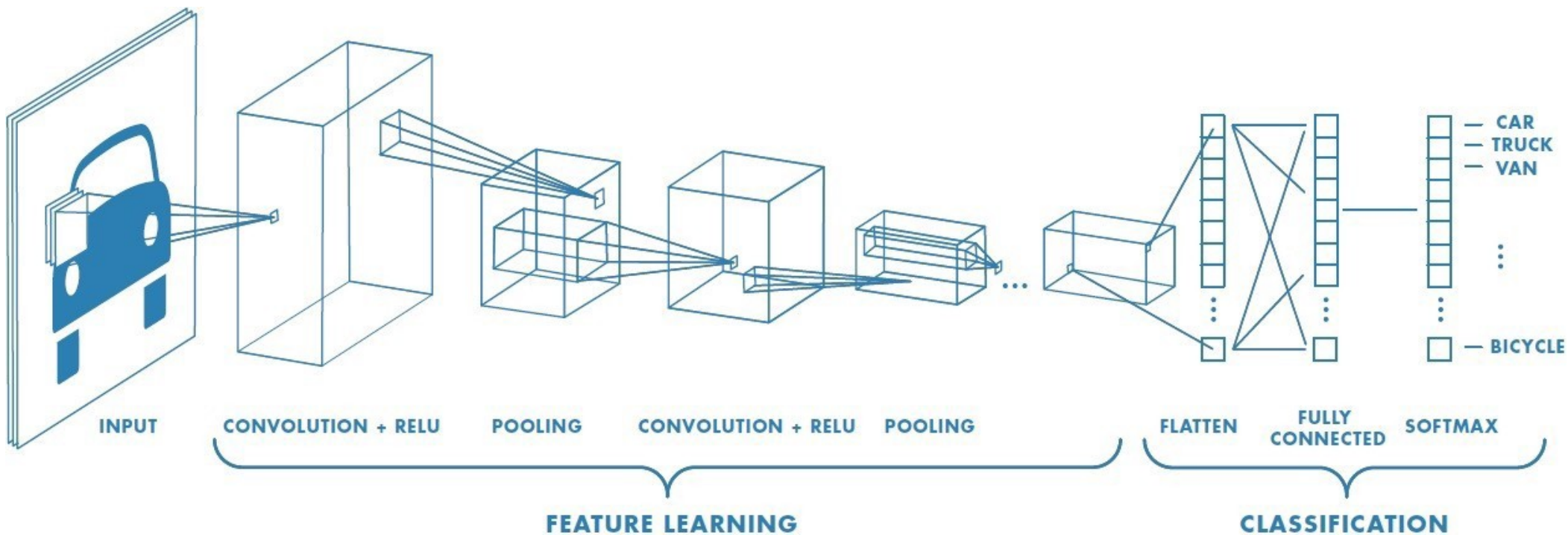
Variantes de descenso por el gradiente (para acelerar/mejorar la convergencia):

- Stochastic gradient descent (SGD): actualizar θ en base al gradiente computado sobre instancias individuales.
- Minibatches: similar a SGD, pero sobre un número (pequeño) de instancias.
- Agregar un término para dar *inercia* a la búsqueda, para pasar por alto mínimos locales.

Deep Learning

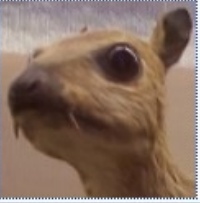
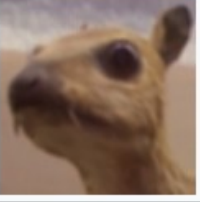


Redes Convolucionales (CNN)

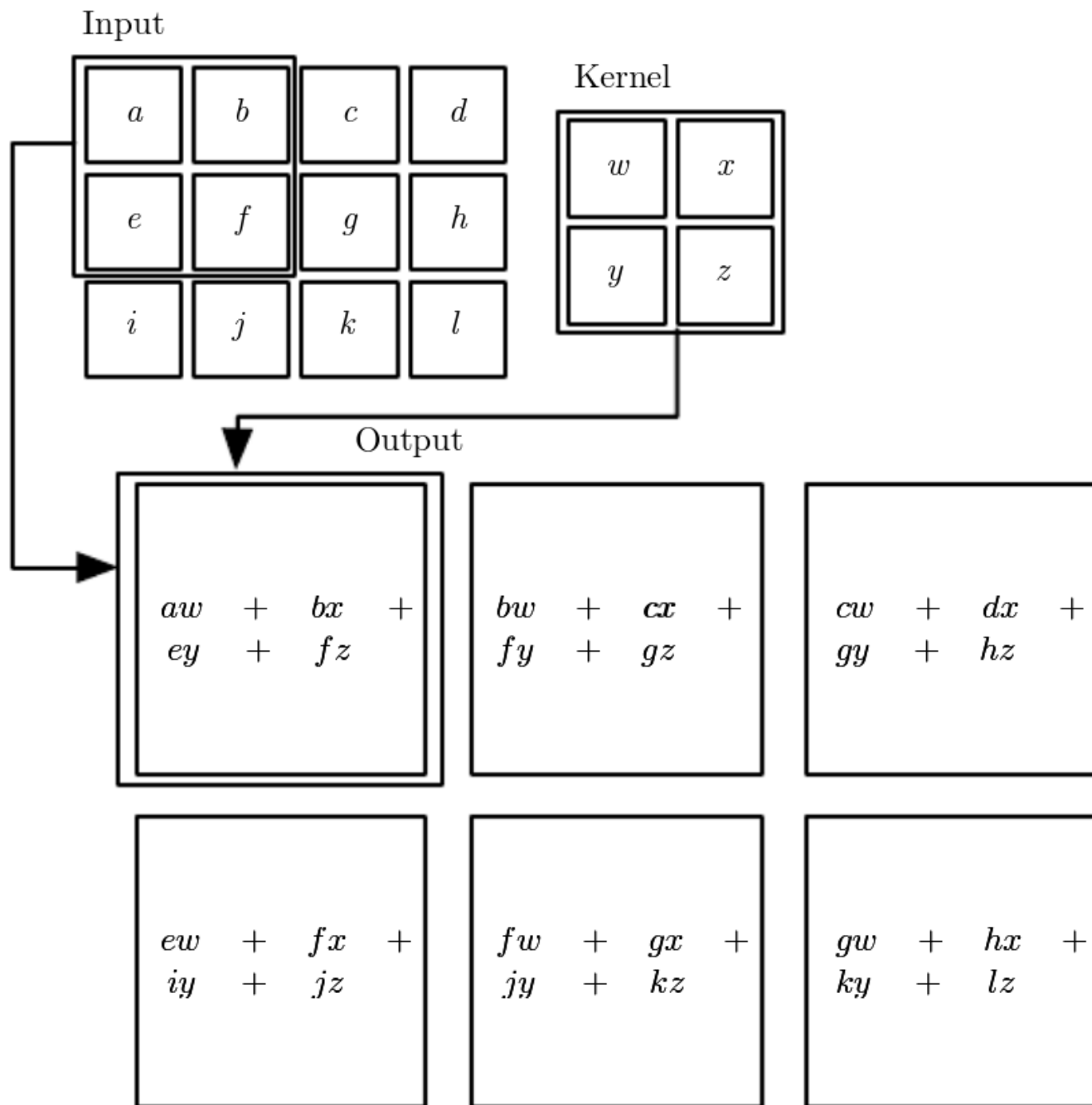


Convolución

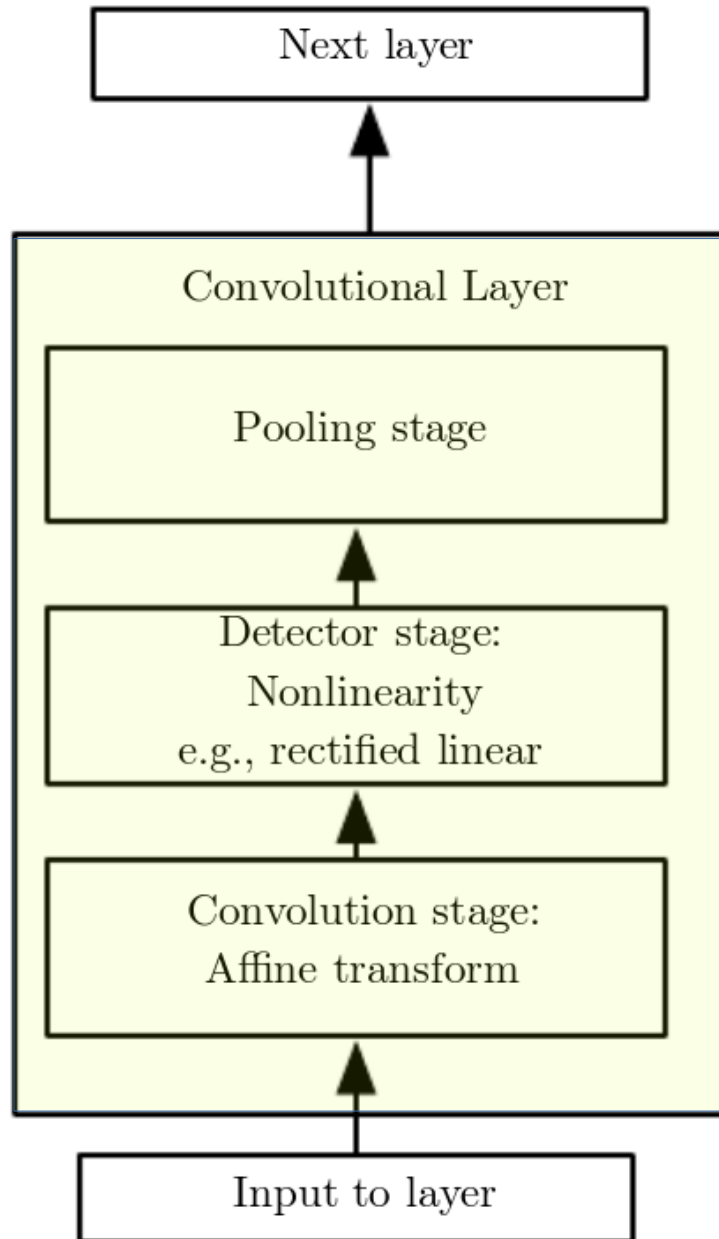
- Matriz de convolución
(o kernel, o máscara)
- Proc. de imágenes
- Hechas a mano; estáticas

Operation	Kernel ω	Image result
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

Convolución

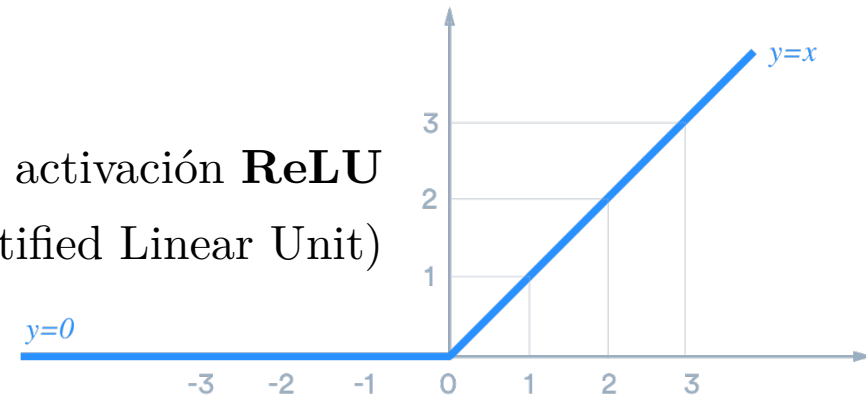


Capa convolucional



[Fuente: Deep Learning Book. Cap.9]

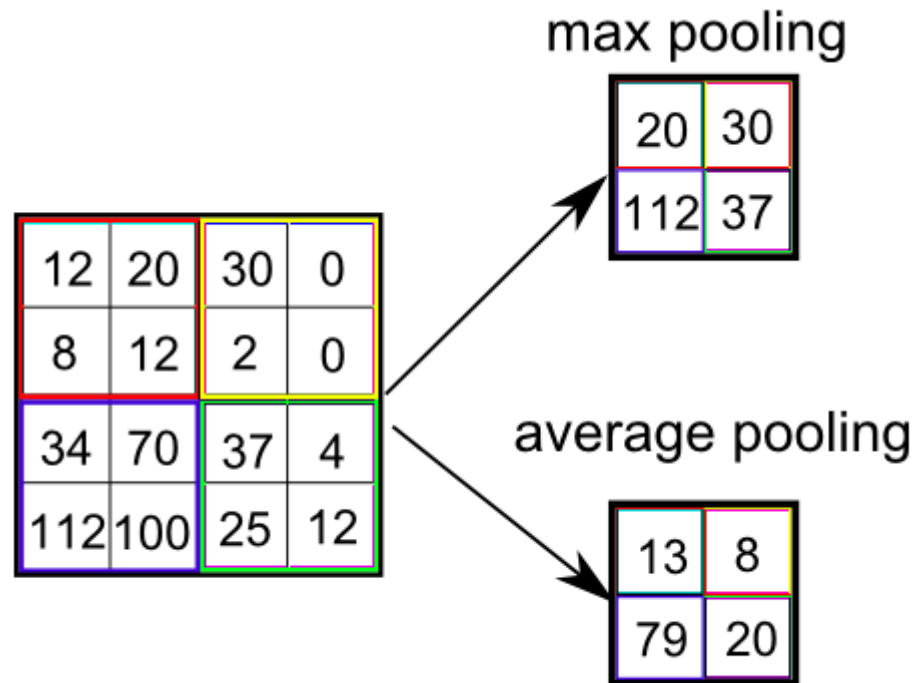
Función de activación **ReLU**
(Rectified Linear Unit)



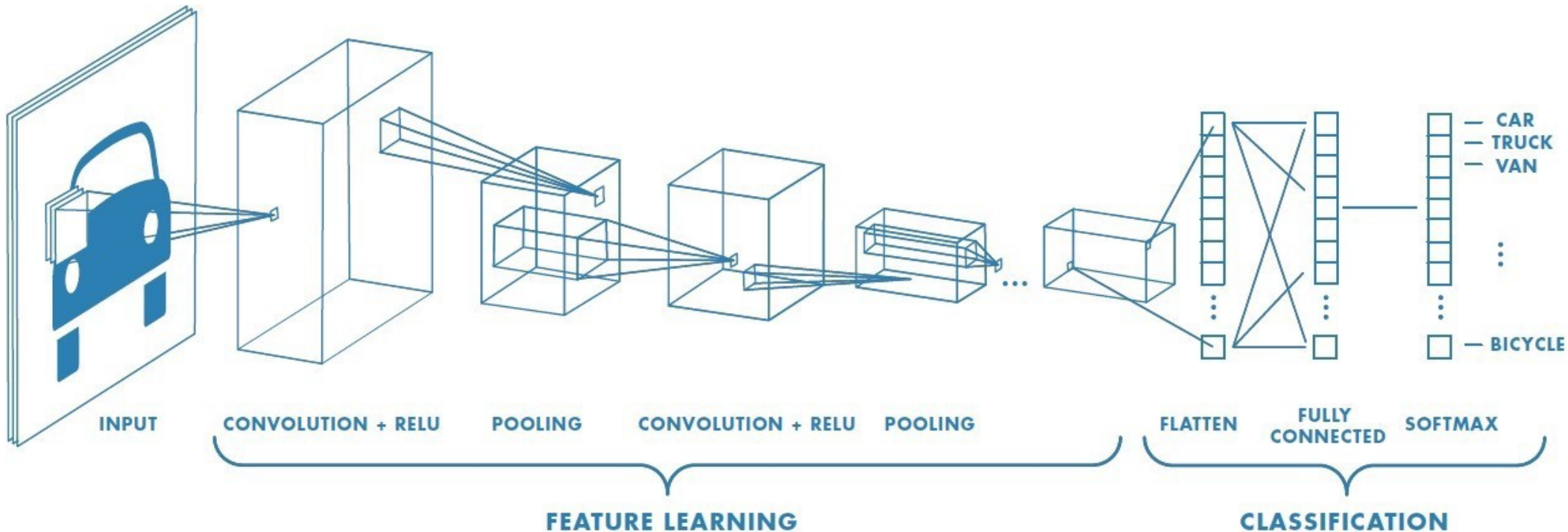
Importante: Los kernels ahora son parte de los parámetros a aprender.

Pooling

- Función para modificar las dimensiones de la salida de una capa



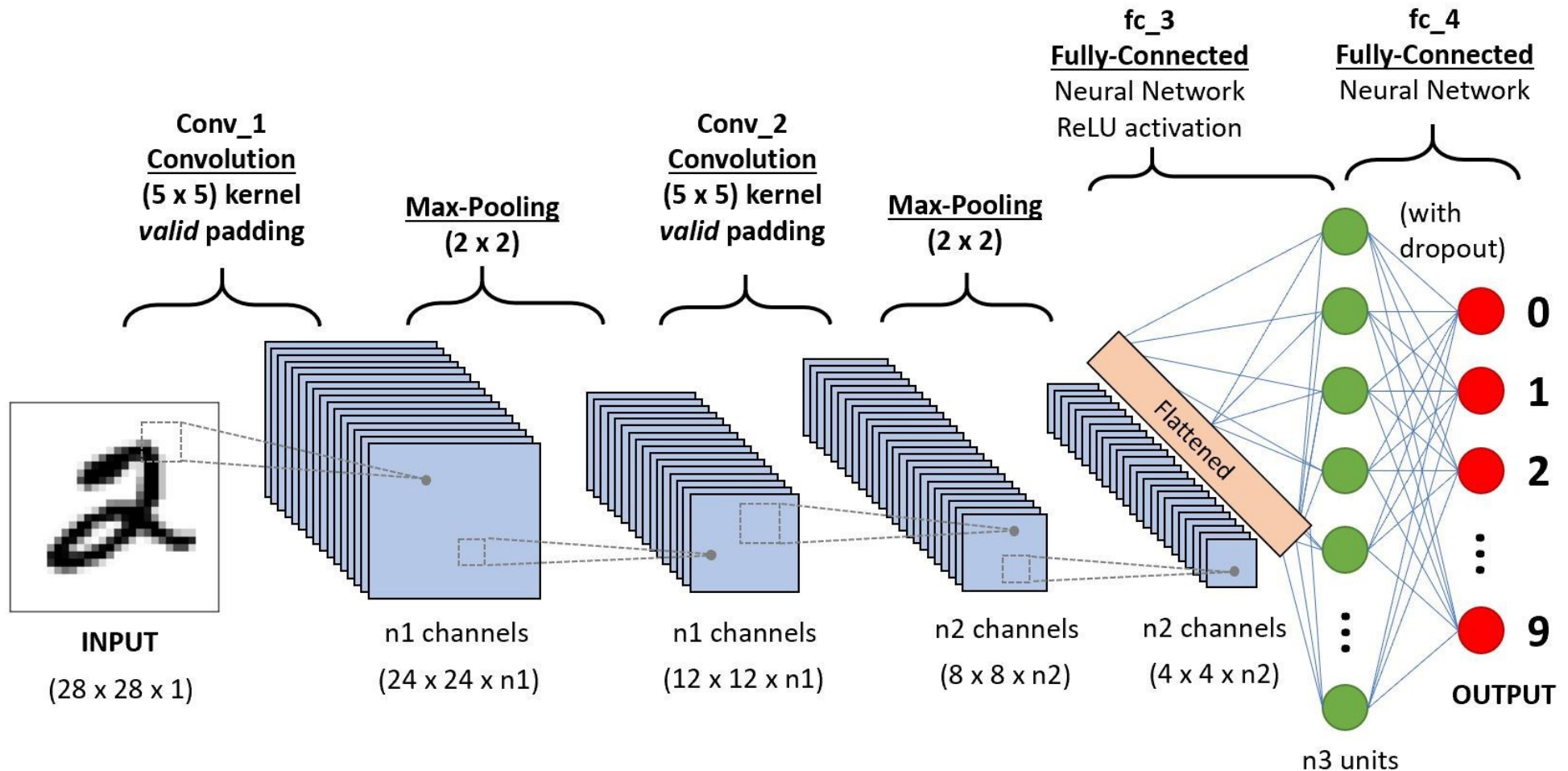
Redes Convolucionales (CNN)



Fuente: "A comprehensive guide to CNN – the ELI5 way" de Sumit Saha --> [Artículo muy recomendable](https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53)

<https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

Redes Convolucionales (CNN)



Fuente: "A comprehensive guide to CNN – the ELI5 way" de Sumit Saha --> [Artículo muy recomendable](https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53)

<https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

Resumen – Redes Neuronales Artificiales

- Regresión lineal simple y múltiple
- Regresión logística
- Ajuste de parámetros: descenso por el gradiente
- Perceptrón simple y multicapa
- Ajuste de parámetros: back-propagation
- Funciones de activación
- Deep Learning: Redes convolucionales (CNN)
- Convolución, pooling, capas densas, softmax